



Fakultät Informatik

**Entwicklung eines VHDL-basierten Applikationsbeispiels
zur Ablösung des FM350-2 Moduls durch das TM FAST
Modul inklusive Integration in das TIA Portal**

Bachelorarbeit im Studiengang Informatik

vorgelegt von

Jan-Eric Gedicke

Matrikelnummer 3643446

Erstgutachter: Prof. Dr. Meitner

Betreuer: Conrad, Maximilian

© 2025

Inhaltsverzeichnis

1	Einleitung	1
1.1	Problemstellung und Motivation	1
1.2	Zielsetzung der Arbeit	1
1.3	Methodisches Vorgehen und Struktur der Arbeit	2
2	Grundlagen	3
2.1	Automatisierungspyramide	3
2.2	Überblick über SIMATIC S7	4
2.3	Aufbau eines SIMATIC S7-Systems	5
2.4	Funktionsweise einer Speicherprogrammierbaren Steuerung	6
2.5	Technologiemodule	7
2.6	Programmierung einer PLC mit TIA-Portal	8
2.7	Software	9
2.8	Maybe	14
3	Analyse der Ablösung des FM350-2 durch das TM FAST Modul	15
3.1	Eigenschaften und Aufbau des TM FAST	15
3.2	Funktionsweise und Limitierungen des FM350-2	23
3.3	Einsatzgebiete der FM 350-2	26
3.4	Vorteile und neue Möglichkeiten durch das TM FAST Modul	28
3.5	Anforderungen an die Neuentwicklung	29
4	Entwicklung des VHDL-basierten Applikationsbeispiels	31
4.1	Konzept und Architektur des neuen Applikationsdesigns	31
4.2	Überlaufkompensation (Dynamic Overflow Compensation)	32
4.3	Analyse der Überlaufkompensation	34
4.4	Umsetzung der Funktionalität in VHDL	38
4.5	Test und Validierung der VHDL-Implementierung	38
4.6	Schnittstelle mit dem TIA Portal	38
4.7	Vergleich mit der bisherigen Lösung	38

5	Fazit und Ausblick	39
5.1	Zusammenfassung der Ergebnisse	39
5.2	Kritische Reflexion und Herausforderungen	39
5.3	Potenzielle Weiterentwicklungen und zukünftige Anwendungen . . .	39
6	Plan	42
7	Fragen	45
8	Wichtig	48
9	TODO	50
10	Tests	52
11	Passwörter	54

1 Einleitung

1.1 Problemstellung und Motivation

Die Produktserie S7-300, insbesondere das FM350-2 Modul, sind nicht mehr in der aktiven Vermarktung und dadurch nicht mehr bestellbar. Das FM350-2 Modul bietet 8 Kanäle für hochkanalige Dosieranwendungen und ist nur noch als Ersatzteil verfügbar. Ein alternatives Lösungskonzept ist der Einsatz des TM FAST Moduls, eines freiprogrammierbaren Hochgeschwindigkeits-Moduls, welches eine erweiterte Kanalanzahl von 10 bis 12 Kanälen ermöglicht. Ziel dieser Bachelorarbeit ist die Entwicklung eines VHDL-basierten Applikationsbeispiels zur Demonstration der Einsatzmöglichkeiten des TM FAST Moduls in hochkanaligen Dosieranwendungen. Gleichzeitig soll die Integration in das TIA Portal erfolgen, um eine reibungslose Nutzung in industriellen Automatisierungsumgebungen zu gewährleisten.

Als dualer Student bei Siemens verfüge ich bereits über Erfahrung in der industriellen Automatisierungstechnik. Diese Arbeit bietet mir die Gelegenheit, mein Wissen in den Bereichen Embedded Systems, Industrieautomation und Software Engineering weiter zu vertiefen. Ich freue mich darauf, dieses praxisnahe Thema in Kooperation mit Siemens zu bearbeiten und einen wertvollen Beitrag zur Weiterentwicklung industrieller Automatisierungslösungen zu leisten.

1.2 Zielsetzung der Arbeit

Ziel dieser Bachelorarbeit ist die Entwicklung eines VHDL-basierten Applikationsbeispiels zur Ablösung des FM350-2 Moduls durch das TM FAST Modul. Dabei soll die Kanalanzahl von 8 auf 10 oder 12 erhöht werden, um eine flexiblere Nutzung in hochkanaligen Dosieranwendungen zu ermöglichen. Ein wesentlicher

Bestandteil der Arbeit ist zudem die Integration des entwickelten Applikationsbeispiels in das TIA Portal, um eine nahtlose Einbindung in bestehende Automatisierungslösungen zu gewährleisten. Neben der Implementierung des VHDL-Codes werden bewährte Methoden des Software Engineerings angewendet, um die Code-Qualität und Skalierbarkeit sicherzustellen. Schließlich soll eine umfassende Dokumentation erstellt werden, die eine detaillierte Beschreibung der Nutzung und Inbetriebnahme des Moduls umfasst.

1.3 Methodisches Vorgehen und Struktur der Arbeit

Zur Umsetzung der Ziele werden zunächst die bestehenden Funktionalitäten des FM350-2 Moduls analysiert und an den aktuellen Erfordernissen der Kunden gespiegelt. Dabei werden sowohl die Hardware- als auch die Software-Architektur untersucht und relevante Funktionen für die Dosieranwendung identifiziert. Anschließend erfolgt die Entwicklung des VHDL-basierten Applikationsbeispiels, wobei die ausgewählten Funktionen des FM350-2 Moduls in VHDL umgesetzt und die Kanalanzahl auf 10 oder 12 erweitert wird. Danach wird die Integration in das TIA Portal vorgenommen, indem das TM FAST Modul an das Automatisierungssystem angebunden und geeignete Schnittstellen sowie Datenstrukturen entwickelt werden. Qualitätssicherungsmaßnahmen spielen ebenfalls eine zentrale Rolle. Hierbei werden bewährte Entwicklungswerkzeuge wie Intel Quartus Prime und das TIA Portal genutzt, Teststrategien zur Funktionssicherung implementiert und eine modulare Softwarearchitektur entworfen. Abschließend wird die Dokumentation erstellt, die eine umfassende Beschreibung der Implementierung und Nutzung des Moduls sowie Anwendungsbeispiele und Testberichte enthält. Die Ergebnisse werden in einer Abschlusspräsentation vorgestellt.

2 Grundlagen

Um die technischen Aspekte und die Funktionsweise der in dieser Arbeit behandelten Systeme zu verstehen, werden im Folgenden die zugrundeliegenden Konzepte und Technologien erläutert. Relevant sind hierbei die Automatisierungspyramide, SIMATIC S7, Speicherprogrammierbare Steuerungen, Quartus und Questa sowie TIA Portal und ?Echtzeitsysteme?.

2.1 Automatisierungspyramide

Die Automatisierungspyramide stellt ein Konzept dar, welches die Hierarchie der Automatisierungstechnik von der Prozess- bis hin zur Unternehmensleitebene beschreibt.

Die Automatisierungspyramide besteht aus insgesamt fünf hierarchischen Ebenen. Diese Ebenen lassen sich in die Systemgruppen IT als Unternehmensleitebene und OT als Fertigungsebene unterteilen [4, S. 4 f.].

Ebenen der OT

- **Ebene 0 – Prozessebene:** Darstellung eines verfahrenstechnischen oder mechanischen Prozesses als unregelmäßiges System.
- **Ebene 1 – Feldebene:** Interaktion mit der Umgebung, Sensorik und Aktorik eines Systems.
- **Ebene 2 – Steuerungsebene:** Koordination von Anlagen und Geräten durch Steuereinheiten wie SPS (PLCs).
- **Ebene 3 – Prozessleitebene:** Überwachung und Ressourcenoptimierung der Gesamtproduktion.

Ebenen der IT

- **Ebene 4 – Betriebsleitebene:** Sammlung und Auswertung von Daten der Prozessleitebene mehrerer Gesamtproduktionen. Koordination zwischen mehreren Abteilungen.
- **Ebene 5 – Unternehmensebene:** Festlegung langfristiger Ziele des Unternehmens mithilfe der erhobenen Daten aus der Betriebsleitebene [9].

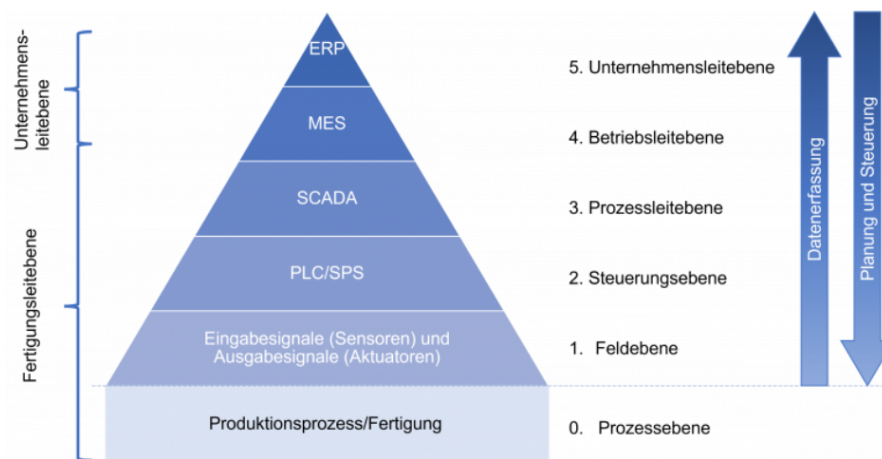


Abbildung 2.1: Automatisierungspyramide, [9]

Das in dieser Arbeit behandelte System findet somit in den Ebenen 1 bis 3 statt. In der Feldebene befindet sich die Sensorik und Aktorik, deren Signale durch eine Steuerungssoftware auf der TM FAST verarbeitet werden sollen. In der Steuerungsebene findet sowohl die Prozessdatenverarbeitung als auch die Kommunikation zwischen dem TM FAST und der PLC statt. Die Visualisierung befindet sich in der Prozessleitebene.

2.2 Überblick über SIMATIC S7

Speicherprogrammierbare Steuerungen – im Englischen „Programmable Logic Controllers“ (PLCs) – wie die SIMATIC S7-Serie von Siemens steuern und regeln unter anderem Maschinen, Antriebe, Ventile, Pumpen und Sensoren. Diese Steuerungen sind frei programmierbar und dadurch universell einsetzbar [[11], S. 146]. Im Folgenden wird ein Überblick über den Aufbau, die Funktionsweise und die Programmierung eines SIMATIC S7-Systems gegeben.

2.3 Aufbau eines SIMATIC S7-Systems

Ein SIMATIC S7-System ist modular aufgebaut und besteht aus mehreren Komponenten. Im Folgenden werden die Hauptkomponenten und ihre Funktionen erläutert.

Controller (CPU)

Die CPU der Steuerung wird auch als Controller bezeichnet und bildet die zentrale Verarbeitungseinheit des Systems. Die Hauptaufgabe der CPU ist es, das Anwenderprogramm auszuführen, das vom Nutzer entwickelt wurde, um spezifische Steuerungs- und Regelungsaufgaben zu übernehmen [[10], S. 25]. Die CPU verarbeitet Signale von Ein- und Ausgabemodulen, führt Berechnungen durch und gibt Steuerbefehle an angeschlossene Geräte, wie Sensoren und Aktoren, aus.

Controller der SIMATIC S7 Serie verfügen über integrierte Kommunikationsschnittstellen, wie Ethernet/PROFINET oder PROFIBUS, die den Datenaustausch mit anderen Steuerungen, Bediengeräten oder übergeordneten Systemen (z. B. SCADA oder MES) ermöglichen [12, S. 14].

Weitere Baugruppen

Um eine Verbindung zu einer Maschine oder Anlage herzustellen, werden Signalbaugruppen verwendet, welche es als Ein- bzw. Ausgabebaugruppen sowohl für Digitale als auch analoge Signale gibt wie Spannungen oder Ströme gibt. Technologiebaugruppen sind Signalverarbeitende Baugruppen, welche Eingehende Signale aufbereiten und entweder direkt an den Prozess oder zur weiteren Verarbeitung an die CPU gegeben. Ein möglicher Einsatzzweck ist z.B. das Zählen von Impulsen, wofür die CPU meist zu langsam ist. Zur Erweiterung der Kommunikationsfähigkeit der CPU bezüglich Protokollen oder Funktionen kommen Kommunikationsbaugruppen zum Einsatz [[10], S. 25].

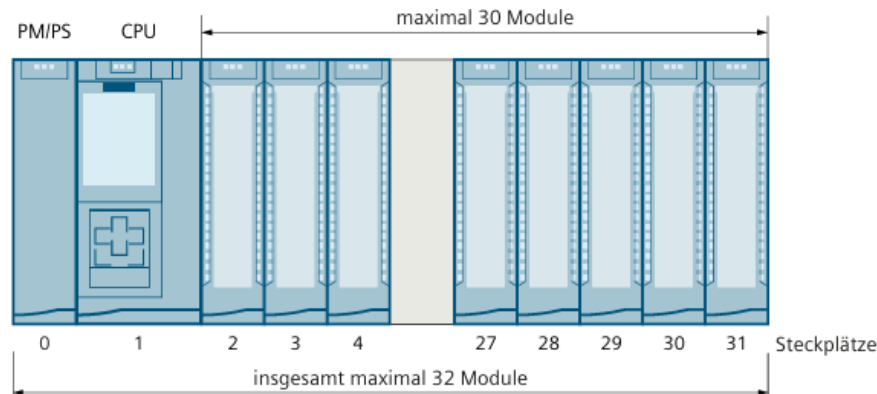


Abbildung 2.2: Aufbau SIMATIC S7-System, [14]

2.4 Funktionsweise einer Speicherprogrammierbaren Steuerung

Grundsätzlich arbeitet eine Speicherprogrammierbare Steuerung nach dem Eingabe-Verarbeitung-Ausgabe-Prinzip (EVA). Der damit verbunden Prozess erfolgt zyklisch und lässt sich wie folgt darstellen:

Über Eingangsbaugruppen werden zunächst die Zustände von angeschlossener Sensorik erfasst. Diese Eingangssignale werden anschließend in einem Prozessabbild gespeichert, welches eine Momentaufnahme der aktuellen Signalzustände darstellt. Änderungen der Signale, die während eines laufenden Zyklus auftreten, werden somit ignoriert [[11], S. 147], [13, S. 39].

Das Prozessabbild dient nun als Datengrundlage für die Verarbeitung durch das Anwenderprogramm. Das Anwenderprogramm ist modular aufgebaut und arbeitet mit sogenannten Bausteinen. Das Hauptprogramm wird durch den Organisationsbaustein1 (OB1) repräsentiert, welcher aus konkreten Anweisungen aber auch Funktions- oder Unterprogrammaufrufen besteht. Die Abarbeitung des OB1 entspricht einem Verarbeitungszyklus, dessen Dauer üblicherweise im zweistelligen Millisekunden Bereich liegt [13, S. 39]. Nach der Verarbeitung des Anwenderprogramms erstellt die SPS ein neues Prozessabbild, das die Ergebnisse der Programmverarbeitung enthält. Dieses Abbild wird schließlich an die Ausgangsbaugruppen übertragen, woraufhin die angeschlossenen Aktoren angesteuert werden. Mit diesem Schritt ist der Zyklus abgeschlossen, und die SPS beginnt unmittelbar mit der Erfassung der neuen Eingangs-Signalzustände für den nächsten Zyklus [[11], S. 147], [13, S. 39].

2.5 Technologiemodule

Die SIMATIC S7-1500 Steuerung ist ein modulares System, das in ihrer Funktionalität frei skalierbar ist. Den Kern der Steuerung bildet die SIMATIC S7-1500 CPU. In ihr wird das Anwenderprogramm ausgeführt und die Verbindung zu anderen Automatisierungskomponenten aufgebaut. Neben der CPU kann das System mit Interface-, Signal-, Kommunikations- und Technologiebaugruppen erweitert werden. Das sogenannte Peripheriesystem ET 200MP ermöglicht den Einsatz der S7-1500 IO-Module entweder, wie beschrieben, direkt an der SIMATIC S7-1500 Steuerung oder an einem entsprechend kompatiblen PROFINET-/PROFIBUS-Teilnehmer, wie z.B. dem Interfacemodul IM 155 PN, das die Daten zwischen Steuerung und Peripheriemodulen austauscht. Abb. 1 zeigt zum einen auf der linken Seite den zentralen Aufbau mit CPU und vier Signalbaugruppen und zum anderen auf der rechten Seite den vergleichbaren dezentralen Aufbau mit Interfacemodul. Die Kommunikation erfolgt über PROFINET. Die Bezeichnung Signalbaugruppe fasst die Digital- und Analogbaugruppen zusammen, die der Steuerung zusätzliche Ein- und Ausgänge zur Verfügung stellen.[7, 8]



Abbildung 2.3: SIMATIC ET 200MP im zentralen und dezentralen Aufbau [14]

Technologiebaugruppen bieten hardwarenahe Signalvorverarbeitung für schnelles Zählen und Messen sowie die genaue Positionserfassung für Inkremental- und SSI-Absolutwertgeber. Bei bisherigen Modulen erfolgt die Parametrierung und Programmierung durch Technologieobjekte oder durch Ansteuerung der IOs im Anwendungsprogramm im TIA Portal. Das aktuelle Portfolio umfasst u.a. das TM NPU, TM Count 2x24 V, TM Posinput 2, TM Timer DIDQ 16x24 V und das TM PTO 4. Ersteres arbeitet mit einem trainierten neuronalen System und es können

verschiedene Sensoren über Gigabit-Ethernet oder USB 3.1 angeschlossen werden. Das TM Count 2x24 V ist eine vielseitig einsetzbare Zählerbaugruppe für Zähl- und Messaufgaben sowie die Positionserfassung mit Inkrementalgebern. Ein weiteres Zähl- und Positionserfassungsmodul ist das TM Posinput 2. Dieses bietet zwei Zählkanäle für Differenz-Eingangssignale nach RS422 mit bis zu 1 MHz und ermöglicht die Positionserfassung mit SSI-Absolutwertgebern. Das TM Timer DIDQ 16x24 V wird für Aufgaben eingesetzt, die einen besonderen zeitlichen Anspruch haben, wie z.B. die exakte Einhaltung von Reaktionszeiten. Das PTO 4 Modul wird zur Ansteuerung von Schrittmotor-Antrieben und Servo-Antrieben mit PTI-Schnittstelle verwendet.[9] Das bisherige Portfolio wird nun mit dem TM FAST erweitert. Dies ist ein Modul, das einen frei programmierbaren FPGA beinhaltet. Im nachfolgenden Abschnitt wird das TM FAST genauer vorgestellt.

2.6 Programmierung einer PLC mit TIA-Portal

Das Totally Integrated Automation Portal (TIA Portal) ist ein Framework für die Projektierung, Programmierung und Inbetriebnahme von SIMATIC S7-Steuerungen und deckt unter anderem Hardware-Konfiguration, Softwareentwicklung und Fehlerdiagnose ab [14, S. 11]. Die Programmierung folgt dabei der internationalen Norm IEC 61131-3, die den Standard für die Softwareentwicklung in speicherprogrammierbaren Steuerungen (SPS) definiert. Diese Norm legt die Struktur und die Programmiersprachen fest, die in Automatisierungsprojekten eingesetzt werden können, um eine einheitliche und herstellerübergreifende Programmierbarkeit zu gewährleisten.

Nach IEC 61131-3 werden fünf standardisierte Programmiersprachen für SPS definiert: KOP (Kontaktplan): Eine grafische Sprache, die sich an elektrischen Schaltplänen orientiert. FUP (Funktionsplan): Eine grafische Sprache zur Darstellung von Logik und Funktionen. AWL (Anweisungsliste): Eine textbasierte Sprache, die ähnlich wie Assembler arbeitet. SCL (Structured Control Language): Eine textbasierte, hochsprachliche Sprache ähnlich zu Pascal. AS (Ablaufsprache): Eine grafische Sprache zur Darstellung von Ablaufsteuerungen [[11], S. 153].

Für die Programmierung im Rahmen der Bachelorarbeit sind insbesondere die Sprachen FUP und SCL relevant.

Funktionsplan (FUP)

FUP ist eine grafische Programmiersprache. Sie basiert auf der Darstellung von Funktionsbausteinen, die durch Verbindungen miteinander verknüpft werden.

Ein typisches Beispiel in FUP ist die Implementierung von UND-, ODER- oder NICHT-Verknüpfungen, bei denen Ein- und Ausgangssignale miteinander verbunden werden. Die Funktionsbausteine enthalten logische Funktionen, mathematische Operationen oder auch spezifische Steuerungsaufgaben, die als grafische Symbole dargestellt sind [15, S. 91 f.].

Structured Control Language (SCL)

SCL ist eine textbasierte Programmiersprache, die der Hochsprache Pascal ähnelt. Sie ist besonders für komplexere Algorithmen und mathematische Berechnungen geeignet. SCL wird in der IEC 61131-3 als Strukturierter Text bezeichnet und unterstützt eine strukturierte Programmierung mit Schleifen, Verzweigungen und Funktionsaufrufen [[11], S. 153].

2.7 Software

Bei der Erstellung einer Anwendung für das TM FAST werden verschiedene Softwareprogramme benötigt. Im TIA Portal wird die Hardwarekonfiguration und das Anwenderprogramm der CPU implementiert. Zudem kann die Firmware über das Engineering-Framework auf das TM FAST geladen werden. Für die Programmierung des Intel® FPGA Cyclone 10 LP wird Intel® Quartus® Prime Lite verwendet. Um die Funktionalität des Hardwareentwurfs vorab zu testen, wird die zugehörige Questa-Intel® FPGA Starter Edition genutzt.

2.7.1 Totally Integrated Automation Portal

Das Totally Integrated Automation Portal (TIA Portal) ist ein Engineering-Framework, das von Siemens für die Anwendung in verschiedensten digitalen Automatisierungsaufgaben entwickelt wurde. Im TIA Portal kann der vollständige Entwicklungsprozess von der Planung, über das integrierte Engineering bis hin zum transparenten Betrieb eines Automatisierungssystems durchgeführt werden. Die Entwicklungsumgebung umfasst unterschiedliche Softwarepakete, wie z.B. SIMATIC

STEP7 für die Programmable Logic Controller (PLC)-Programmierung, SIMATIC WinCC für die Visualisierung von Maschinenprozessen auf Human Machine Interfaces (HMIs), SINAMICS Startdrive zur Parametrierung von Antrieben oder SIMATIC Energy Suite für die Erfassung von Energiedaten und Überwachung des Energieverbrauchs in der Anlage. In dieser Arbeit wird TIA Portal V20 verwendet [[12], [13]]. Damit die CPU eine Verbindung zu dem Technologiemodul aufbauen kann, ist eine Hardwarekonfiguration im TIA Portal notwendig. Hierfür werden die benötigten Komponenten anhand ihrer Artikelnummer ausgewählt und miteinander verbunden. Abb. 6 zeigt die Hardwarekonfiguration im TIA Portal mit einem Powermodul (PM) zur Spannungsversorgung (Slot 0), einer CPU 1516-3 PN/DP (Slot 1) und einem TM FAST (Slot 2). Da die drei Komponenten auf einer gemeinsamen Schiene angebracht sind, können sie über den Rückwandbus kommunizieren. Es ist keine zusätzliche Verbindung über PROFINET oder PROFIBUS erforderlich.

Falsches bild



Abbildung 2.4: Hardwarekonfiguration im TIA Portal mit TM FAST

Das TM FAST besitzt nach der Hardwarekonfiguration im TIA Portal einen eigenen Eintrag. Hier können unter Online & Diagnostics allgemeine Informationen über das Modul ausgelesen, der Diagnose-Status abgefragt und eine neue Firmware auf das FPGA geladen werden. Unter dem Menüpunkt Commissioning kann

der Status und die aktuelle User-Logik Version abgefragt werden. Zudem gibt es die Möglichkeit, verschiedene Kommandos an das FPGA zu schicken, wie z.B. das Laden der TM FAST-Anwendung aus dem Flash-Speicher oder die Aktivierung der Debug-Schnittstelle. Die Register der azyklischen Kommunikation können unter dem Punkt TM FAST user data records manuell beschrieben oder ausgelesen werden. Die Funktionsmöglichkeiten zur Steuerung der Anwendung, Lesen und Schreiben von Daten sowie Laden der Anwendung können entweder im Menü des TIA Portals oder als Funktionsbausteine (FB) der TIA-Anweisungsbibliothek LTMFAST im Programm ausgeführt werden. Die TIA-Library enthält die Anweisungen LTMFAST ControlREC, LTMFAST UserReadREC, LTMFAST UserWriteREC und LTMFAST AppDownload [6, 17].

2.7.2 Intel® Quartus® Prime

Die Intel® Quartus® Prime Software ist eine Designumgebung für Intel® FPGAs. Es beinhaltet alle Schritte der Entwicklung von Designerfassung, Synthese bis hin zur Optimierung, Verifizierung und Simulation. Von Intel® werden drei verschiedene Versionen angeboten. Neben der kostenlosen Lite Edition gibt es noch die Standard und Pro Edition. Die Versionen unterscheiden sich teilweise im Funktionsumfang sowie in der Unterstützung der FPGA Board Familien. Das FPGA Cyclone 10 LP, das im TM FAST verbaut ist, kann nur in den Editionen Standard und Lite verwendet werden. Für die Umsetzung der Anwendungen in dieser Arbeit wird die Quartus® Prime Version 22.1std.0 Lite Edition verwendet [18]. Das Quartus Projekt, das zur Programmerstellung für das TM FAST dient, kann in verschiedene Revisionen unterteilt werden. Diese beinhalten unabhängig voneinander eine eigene VHDL-Architektur sowie eine Konfigurationsdatei. Die Architektur bestimmt das Verhalten des Programms. In der Konfiguration können Parameter wie z.B. die Frequenz der User-Clock oder der Schnittstellentyp der acht Channel individuell eingestellt werden. Über einen Multiplexer wird anschließend die ausgewählte Architektur der Top-Entity TFL FAST USER e zugeordnet. Die Aufteilung in verschiedene Revisionen ermöglicht es, mehrere Anwendungen zu implementieren, ohne das Programmgerüst aus System- und User-Logik jedes Mal erneut aufzusetzen.

In der Quartus® Prime Software werden mehrere Schritte durchgeführt, um vom geschriebenen VHDL-Code zum fertigen Bitstream, der die Logik der Anwendung

für das Technologiemodul beinhaltet, zu gelangen. Der Kompilationsvorgang beginnt mit der Analyse und Synthese. Es wird die logische Vollständigkeit und Konsistenz des Projektes sowie Syntaxfehler geprüft. Anschließend wird die Logik der Dateien der Design Entity synthetisiert und ein Technologie-Mapping durchgeführt. Das bedeutet, es werden Flipflops, Latches und Zustandsautomaten aus dem VHDL-Code abgeleitet und optimiert, sodass der Ressourcenverbrauch minimiert werden kann. In der Analyse und Synthese werden Algorithmen angewandt, um die Anzahl der Gatter zu reduzieren und redundante Logik zu entfernen. Dadurch wird gewährleistet, dass die Architektur des Geräts so effizient wie möglich genutzt werden kann. Nach der Analyse und Synthese folgt der Fitter. Hier werden die logischen und zeitlichen Anforderungen des Projekts mit den verfügbaren Ressourcen des Zielgeräts abgeglichen. Jeder Logikfunktion wird der beste Logikzellenplatz hinsichtlich des Routings und Timings zugeordnet und geeignete Verbindungspfade und Pinbelegungen ausgewählt [19].

Ein FPGA besteht aus einer großen Anzahl von programmierbaren Logik-Blöcken, die jeweils eine kleine Menge an digitaler Logik und programmierbaren Routing implementieren können. Routing ermöglicht die Verbindung der Ein- und Ausgänge der Logikblöcke zu einer größeren Schaltung. Die Verzögerung einer im FPGA implementierten Schaltung ist hauptsächlich auf die Verzögerung beim Routing zurückzuführen. Zudem wird der größte Anteil der Fläche eines FPGAs dem programmierbaren Routing zugeschrieben [20]. Nach dem Fitter kommt das Assembler-Modul zum Einsatz. Dieses fügt die Post-Fit-Logikpartitionen zusammen. Es entstehen Dateien, die zur Programmierung oder Konfiguration eines FPGAs verwendet werden können. Der Assembler konvertiert automatisch die Baustein-, Logikzellen- und Pin-Zuweisungen des Fitters in ein sogenanntes Programmier-Image in Form eines Programmer Object Files (.pof) oder SRAM Object Files (.sof). Für den Prozess, die Logik auf das TM FAST zu laden, wird ein Raw Binary File (.rbf) benötigt. Dies ist ein Bitstream, der vom Assembler aus den Speicherabbilddateien (Memory Image Files .mif) generiert wird [19]. Die weiteren Schritte des Verfahrens, die in VHDL hinterlegte Anwendungslogik auf das TM FAST zu importieren, wird in Abschnitt 2.3.4 nochmal ausführlicher betrachtet.

In der Quartus® Prime Software gibt es mit dem Tool Signal Tap Logic Analyzer eine Möglichkeit den Hardwareentwurf zu debuggen. Die Software erfasst und zeigt das Echtzeit-Signalverhalten in einem Intel® FPGA Design an. Es kann das Verhalten interner Signale während des Normalbetriebs untersucht werden, ohne zu-

sätzliche Peripherie anschließen zu müssen. Durch Bedingungen können der Start und das Ende der Datenerfassung bestimmt werden. Zudem gibt es die Option eine Triggerbedingung festzulegen. Die aufgenommenen Daten können anschließend abgespeichert und gefiltert werden [21]. Um das Softwaretool nutzen zu können sind die Hardware-Komponenten Intel® FPGA Download Kabel (USB-Blaster) und der TM FAST Debug Connector notwendig. Diese bilden das Debug-Interface zwischen PC und TM FAST. Der Connector wird auf dem Modul angebracht und stellt somit eine Verbindung zur JTAG-Schnittstelle her [10].

2.7.3 Questa-Intel® FPGA

Es gibt zwei Software-Editionen zur Simulation von Hardware designs. Die kostenpflichtige Questa-Intel® FPGA Software und die Questa-Intel® FPGA Starter Edition. Beide sind eine Version der Siemens EDA Questa Core Software für Intel® FPGAs [22]. Questa bietet eine Simulations-, Debug- und Verifikationsplattform zur Validierung von FPGA-Entwürfen [23]. Die Simulation kann entweder über die Quartus® Prime Software oder direkt in der Questa-Softwareplattform mit einem Simulationsskript gestartet werden. Um eine VHDL-Architektur testen zu können, wird eine Testumgebung, auch Testbench genannt, benötigt. Es werden die Eingangssignale (Stimuli) nachgebildet, um die infolgedessen generierten Ausgangssignale überprüfen zu können. Die Stimuli werden in verschiedenen Prozessen generiert und mit der Instanz des zu testenden Modells verbunden [13]. Ein Prozess kann beispielsweise für die Nachbildung des Clock-Signals zuständig sein. Dies erfolgt durch eine Dauerschleife mit einer Wartezeit, nach deren Ablauf der Signalwert getoggelt wird. Dadurch ergibt sich eine Clock mit der Periode der doppelten Wartezeit. Anschließend kann die Simulation in einem Zeitdiagramm analysiert werden. Abb. 7 zeigt den zeitlichen Verlauf einer Testbench, die den Funktionsablauf der Signum-Funktion überprüft. Eine Signum-Funktion gibt bei einer positiven Zahl den Wert “+1“, bei null den Wert “0“ und bei einer negativen Zahl den Wert “-1“ aus [24]. In der Testbench werden drei signed Werte im Hexadezimal-Format abwechselnd als Input verwendet. Bei einer steigenden Flanke der Clock liegt nach einem Wechsel des Eingangssignals das Ergebnis am Ausgang an.

Falsches bild



Abbildung 2.5: Zeitdiagramm zur Simulation der Signum-Funktion in einer Testbench in der Questa-Intel® FPGA Starter Edition

2.8 Maybe

-Echtzeitsysteme? übergang zu was ich mache (ist das wichtig? wahrscheinlich nicht) -verwendete Software:Questa, Quartus, TIA ... (Bei Umsetzung mit 3 Sätzen)

3 Analyse der Ablösung des FM350-2 durch das TM FAST Modul

In diesem Abschnitt werden die technischen Grundlagen und Eigenschaften der beiden betrachteten Module, FM350-2 und TM FAST, vorgestellt. Es werden deren Aufbau, Funktionsweise, Schnittstellen und typische Einsatzgebiete erläutert. Ziel ist es, die relevanten Technologien und Unterschiede beider Systeme darzustellen, um die Basis für die spätere Analyse der Ablösung des FM350-2 durch das TM FAST Modul zu schaffen.

3.1 Eigenschaften und Aufbau des TM FAST

Das TM FAST bietet die Möglichkeit, eigene Anwendungen in das SIMATIC S7-1500 Technologiemodul einzubetten. Hardwareseitig besteht es aus einem Backend-Baustein, der die Siemens TM FAST-Firmware bereitstellt, und einem Frontend-Baustein (Intel® Cyclone 10 LP FPGA), der die sogenannte TM FAST-Anwendung hostet. Diese Applikation bestimmt die Hardwarefunktionalität des Technologiemoduls und kann durch applikationsspezifisches Programmieren des FPGAs selbst definiert werden. Hierzu wird die externe Software Intel® Quartus® Prime verwendet. Näheres zu den Programmen der Softwareentwicklung und -simulation folgt in Abschnitt 2.3. [Guillaume: Specification for TM Fast, 10]

Eine vergleichbare Baugruppe ist der High Speed Boolean Processor FM352-5 aus dem Automatisierungssystem S7-300. Dieser wird mit einer Zykluszeit von 1 μ s für Anwendungen verwendet, die eine schnelle Binärsteuerung sowie Schaltvorgänge benötigen. Die Funktionsbaugruppe wird in STEP 7 mit Kontaktplan (KOP) oder Funktionsplan (FUP) mit einem zur Verfügung stehenden Anweisungssatz programmiert [11]. Durch die Programmierung des FPGAs des TM FAST in der

Intel® Quartus® Prime Software ergibt sich eine höhere Flexibilität in der Anwendungsprogrammierung und ermöglicht spezifische und benutzerdefinierte Lösungen. Mit einer Zykluszeit von 20 ns und die somit geringere Verarbeitungs- und Reaktionszeit eröffnen sich neue Einsatzmöglichkeiten. Es können beispielsweise beliebige Pulsmuster an den Ausgängen erzeugt werden, die zeitlich sehr fein aufgelöst werden können (unter 1 μ s) und in Abhängigkeit.

Im Gegensatz zur einfachen Verarbeitung digitaler Signale kann mit dem TM FAST auch die Auswertung anderer Signale, wie z. B. eines Zählstands, erfolgen. Nachfolgend sind weitere Beispiele aufgelistet. [[7], [8]]

- Positionsabhängige Nocken
- Sehr schnelle Reaktionen auf digitale Ereignisse
- Zählen von Signalen bis zu 5 MHz
- Erfassungslogiken für Inkremental- und Absolutwertgeber mit Ableitung einer unmittelbaren Reaktion
- Präzise Ansteuerung sowie flexible Steuerung von Lasern in verschiedenen Anwendungsfällen
- Pixelgenaue Punktmuster auf mehreren parallelen Spuren
- Exakte Dosieraufgaben
- Durchführung von Berechnungen und Vorverarbeitungen innerhalb des Moduls, z. B. zur Fehlerfrüherkennung

Aufgrund der freien Programmierbarkeit können einzelne Funktionen variabel angepasst und kombiniert werden. Zusammenfassend eignet sich das TM FAST für alle Aufgaben, bei denen die speicherprogrammierbare Steuerung (SPS) mit Peripheriebaugruppe entweder nicht schnell genug ist oder die nicht durch ein vorhandenes Technologiemodul umgesetzt werden können und daher einer speziellen Einzellösung bedürfen.

Der **Aufbau** des TM FAST aus Backend-Gerät, Frontend-Gerät, Prozessschnittstelle sowie die Anbindung an die CPU und Peripherie wird in Abb. 2 veranschaulicht.

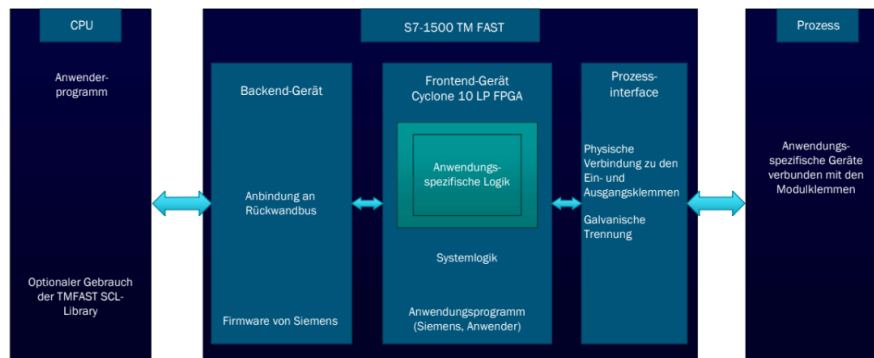


Abbildung 3.1: Abbildung 6: Aufbau des TM FAST als Blockdiagramm [Guillaume: Specification for TM Fast]

Das **Backend-Gerät** realisiert die Verbindung des TM FAST zu anderen SIMATIC

Komponenten über den Rückwandbus der zugehörigen SIMATIC S7-1500 Automatisierungseinheit. Die entsprechende Firmware wird von Siemens bereitgestellt und gewartet. Diese regelt u.a. die Kommunikation mit der S7-1500 Steuerung, den zyklischen und azyklischen Datentransfer sowie den Diagnosestatus. Genaues zur Kommunikation folgt im späteren Verlauf des Abschnittes. Des Weiteren wird im Backend-Gerät die TM FAST-Applikation aus dem Flash geladen, über die SPI-Schnittstelle auf das FPGA übertragen, validiert und nach Bestätigung aktiviert. Zudem wird das Frontend-Gerät beobachtet, um Fehlverhalten zu erkennen. Sind die Parameter des Modules gesetzt, wird durch die Firmware die Systemlogik entsprechend angepasst. Das Backend-Gerät liest die Temperatur des Moduls über den Temperatursensor via I2C aus und kontrolliert alle LEDs des TM FAST über SPI. [Guillaume: Specification for TM Fast]

Das **Frontend-Gerät** bildet den Kern des TM FAST. Hier wird die anwendungsspezifische Software abgelegt und ausgeführt. Das FPGA-Programm, bzw. die TM

FAST-Anwendung, besteht aus zwei Teilen, die FAST-Systemlogik (TFL FAST SYS) und der User-Logik (TFL FAST USER). Die Programmierung erfolgt in der Hardwarebeschreibungssprache Very High Speed Integrated Circuit Hardware Description Language (VHDL). Der Entwurf der Anwendungslogik ist in das System eingebettet und besitzt eine Schnittstelle zur Systemlogik in der Form der VHDL Top Entity TFL FAST USER e [Guillaume: Specification for TM Fast]. Über diese

können die Ein- und Ausgänge abgefragt bzw. gesteuert werden und die Kommunikation mit der CPU stattfinden. Ein VHDL-Programm besteht grundsätzlich aus den eingebundenen Packages bzw. Bibliotheken, der Entity, die als Schnittstellenbeschreibung fungiert, einer Architektur, die das Verhalten des Programmes beschreibt und einer Konfiguration, in der die entsprechende Architektur ausgewählt wird und gegebenenfalls Submodule und Parameter hinzugefügt werden [13]. Das spezifische Anwendungsprogramm wird somit in der VHDL-Architektur mit der Top Entity TFL FAST USER e implementiert. In Abb. 3 wird der Zusammenhang zwischen den beiden Logiken und die Einbindung in das System mit Backend-Gerät und Peripherie veranschaulicht.

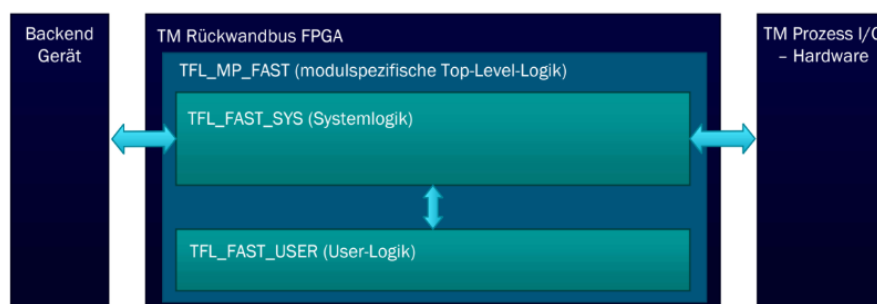


Abbildung 3.2: Abbildung 7: Aufbau aus Backend- und Frontend-Gerät und Zusammensetzung der Logik des TM FAST [Guillaume: Specification for TM Fast]

Als Frontend-Gerät wird das Intel® FPGA der Familie Cyclone 10 LP verwendet. Der exakte Gerätenamen lautet 10CL025YU256A7G. Die maximal verfügbaren **Ressourcen des FPGAs** sind in der nachfolgenden Tab. 1 aufgelistet.

Ressource	Anzahl
Logikelemente	24 624
M9K Memory Blöcke	66
18 x 18 Multiplizierer	66
PLL	4
Clocks	20
Maximale I/O	150

Tabelle 3.1: Ressourcen des Intel® FPGA Cyclone 10 LP [15]

Ein Logikelement (LE) ist die kleinste Logikeinheit in der Intel® Cyclone 10 LP

Gerätearchitektur. Jedes LE hat vier Eingänge, eine Look-Up Tabelle mit vier Eingängen, ein Register und eine Ausgangslogik. Die Look-Up Tabelle ist ein Funktionsgenerator, der jede Funktion mit vier Variablen implementieren kann. Der eingebettete Memory-Speicher ist aus M9K Memory-Blöcken aufgebaut. Jeder dieser Blöcke stellt 9 Kbit an On-Chip Speicher zur Verfügung. Diese M9K-Blöcke können beliebig miteinander verschachtelt und kombiniert werden, um größere Speicherplätze zu generieren. Insgesamt ergibt sich aus den 66 Blöcken eine Speicherkapazität von 594 Kbit, das umgerechnet 608256 Bit entspricht. Die Memory-Blöcke können als RAM, FIFO-Puffer oder ROM konfiguriert werden. Jeder eingebettete Multiplizierer-Block kann entweder als einzelner 18x18-Bit oder zwei individuelle 9x9-Bit Multiplizierer initialisiert werden. Die Funktionsweise der Blöcke kann entweder über die Parametrierung bestimmter IP-Cores in der Intel® Quartus® Prime Software definiert werden oder direkt über den VHDL oder Verilog HDL Code implementiert werden. Dem FPGA stehen vier Phasenregelschleifen (PLL) und bis zu 20 Clocks zur Verfügung. Die PLLs können dynamisch rekonfiguriert werden, wodurch die Phase und Frequenz der Clock geändert werden kann. Die General Purpose I/Os (GPIOs) des FPGAs bieten eine Vielzahl an Einstellmöglichkeiten, wie z.B. einen Pull-Up Widerstand oder eine programmierbare Anstiegsgeschwindigkeitsregelung zur Optimierung der Signalintegrität.[14] Nach Erläuterung des Aufbau- und Hardwarekonzepts des TM FAST folgt im restlichen Teil des Abschnittes der **äußere Aufbau** (siehe Abb. 4), die **Pinbelegung** der Klemmen, die die Anbindung von Peripheriegeräten ermöglichen sowie die genauere Erklärung der **Kommunikationsmöglichkeiten** zwischen FPGA und CPU. Das Modul besteht aus einer Baugruppe der Bauform MP, einem S7-1500 Frontstecker mit 40 Klemmen und vier zusätzlichen Klemmen zur Spannungsversorgung sowie einem TM FAST Debug Connector zur direkten Verbindung mit dem

FPGA. Dieser dient für die Durchführung von Service- und Entwicklungsarbeiten, unabhängig von der Frontstecker-Verdrahtung. Abb. 4 zeigt das Technologiemodul mit Zuordnung der äußeren Kennzeichnungen und Anzeigeeinheiten, die Erläuterung erfolgt dazu in Tab. 2. An der Außenseite kann man u.a. den Modultyp sowie -bezeichnung TM FAST, die Artikelnummer 6ES7554-1AA00-0AB0, den Funktionsstand und den Firmware-Stand V1.0.0 ablesen. Des Weiteren wird mit LEDs der Status der Ein- und Ausgänge, der Versorgungsspannung und der Diagnose angezeigt.

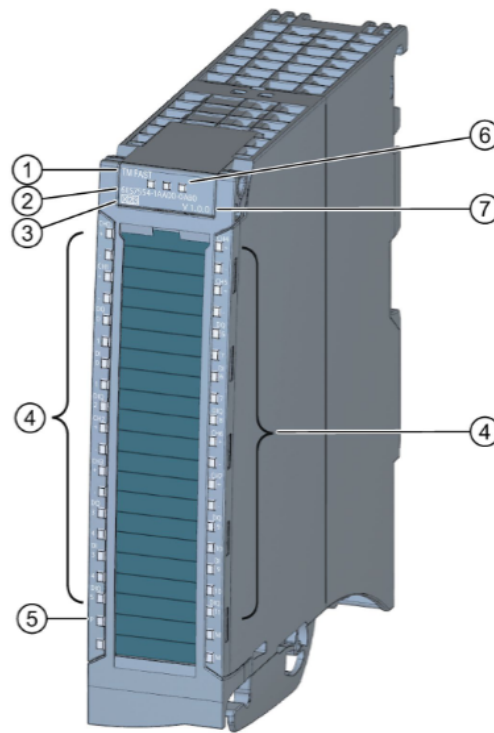


Abbildung 3.3: Abbildung 8: Technologiemodul TM FAST [8]

Num.	Bezeichnung
1	Modultyp und -bezeichnung
2	Artikelnummer
3	Funktionsstand
4	LEDs für Status der Ein- und Ausgänge
5	LED für Versorgungsspannung
6	LEDs für Diagnose
7	Firmware-Stand

Tabelle 3.2: Zuordnung der äußeren Kennzeichnungen und Anzeigeeinheiten gemäß Abb. 8

Um externe Signalgeber oder -empfänger mit dem TM FAST verbinden zu können, wird am Modul ein **40-poliger Frontstecker** angebracht. Dieser ist in Abb. 5 dargestellt. In Tab. 3.1 werden die jeweiligen Klemmen ihrer Funktion zugeordnet.

Das TM FAST umfasst insgesamt je acht 24 V-Digitaleingänge (DI) und -ausgänge (DQ) mit einem Ausgangsstrom von 0,1 A und einer maximalen Schaltfrequenz

von 200 kHz. Des Weiteren sind vier 24 V-Digitalein-/ausgänge (DIQ) mit 0,3 A Ausgangsstrom und maximal 1 kHz Schaltfrequenz verfügbar. Diese können in der Software als Ein- oder Ausgang konfiguriert werden. Die Digitalausgänge sind gegen Überlast und Kurzschluss geschützt.

Neben den digitalen Ein- und Ausgängen gibt es acht RS422/485-Kanäle (CH+ und CH-), die auch als TTL- (5 V) Kanäle eingesetzt werden können. Die Richtung ist wie bei den DIQ-Kanälen einstellbar. Wird mit einem RS485-Signal gearbeitet, so wird jeweils ein Leitungspaar verwendet. Bei einem Signal mit TTL-Standard wird nur eine einzelne Leitung benötigt.

Um kurze Störungen oder instabile Signale auf der Leitung zu filtern, kann für die Digitaleingänge und Kanäle eine Eingangsverzögerung parametrisiert werden. Den DIs kann eine Eingangsverzögerung im Bereich von 0 bis 20 ms in Schritten von 0,001 ms fest in den Systemlogik-Parametern eingestellt werden. Die gleiche Möglichkeit besteht für die RS422/RS485/TTL-Kanäle. Hier kann die Eingangsverzögerung im Bereich von 0,1 μ s bis 1000 μ s festgelegt werden.

Neben der festen Parametrierung kann die Verzögerung auch in der TM FAST-Anwendung frei programmiert werden [8].

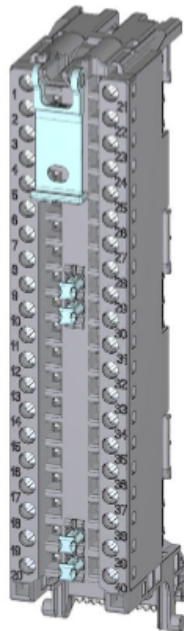


Abbildung 3.4: Abbildung 9: Frontstecker des TM FAST [14]

Klemmen	Anschlussbelegung
1..4, 10..13, 21..24, 30..33	RS422/RS485/TTL-Ein- oder Ausgangssignal
5, 6, 14, 15, 25, 26, 34, 35	Digitalausgang
7, 8, 16, 17, 27, 28, 36, 37	Digitaleingang
9, 18, 29, 38	Digitalein-/ausgang
19, 20, 39, 40	Masse

Tabelle 3.3: Anschlussbelegung des Frontsteckers aus Abb. 9

Die **Kommunikation** zwischen CPU und TM FAST kann, wie bereits erwähnt, entweder zyklisch oder azyklisch erfolgen. Die zyklischen Ein- und Ausgangsdaten sind unterteilt in acht Doppelwörter. Ein Doppelwort besteht aus 32 Bit, sodass in Summe mit 32 Byte kommuniziert werden kann. Der zyklische Datenaustausch wird in der Regel aus der Sicht der CPU betrachtet. Das bedeutet, dass die Steuerschnittstelle (CTRL IF) die Ausgangsdaten, die von der CPU an das TM gesendet werden, beinhaltet. Entsprechend werden die Eingangsdaten, die die CPU aus dem TM FAST liest, als Rückmeldeschnittstelle (FB IF) bezeichnet. Auf Seiten des TMs ist die Verwendung der Schnittstellen folglich genau entgegengesetzt. In der Umsetzung der Laser-Scanner-Anwendung in Abschnitt 3.1 erfolgt der Datenaustausch rein über die zyklische Schnittstelle zwischen CPU und Technologiemodul. Der Grund hierfür ist, dass somit der Zeitpunkt des Datentransfers genau bestimmt werden kann und mit jedem Zyklus der CPU neue Werte übermittelt werden können. Die azyklische Datenmenge kann für eine Anwendungslogik konfiguriert werden. Diese ist dann für diese Logikversion fest hinterlegt und kann während der Ausführung nicht mehr geändert werden. Es kann zwischen 4, 32, 64 und 128 Byte für die azyklische Kommunikation gewählt werden. Es steht je ein Register mit der gewählten Größe für den Lese- und Schreibprozess zur Verfügung. Der Datenaustausch erfolgt über die Anweisungen LTMFAST UserWriteREC (Schreibzugriff) und LTMFAST UserReadREC (Lesezugriff) im TIA Portal (Abschnitt 2.3.1). Es können verschiedene Parameter genutzt werden, die den Ablauf des Informationsaustausches koordinieren und steuern. [[?], [8]] Nach Begutachtung der Eigenschaften des TM FAST, den Aufbau aus verschiedenen Hardwarekomponenten, die entsprechenden technischen Daten und die Anbindung an das SIMATIC S7-1500

Automatisierungssystem wird im nächsten Abschnitt die benötigte Software zur Programmierung des Technologiemoduls genauer betrachtet.

3.2 Funktionsweise und Limitierungen des FM350-2

Neu:

Die Funktionsbaugruppe FM 350-2 ist eine 8-kanalige Zählerbaugruppe mit integrierter Dosierfunktion für das Automatisierungssystem S7-300. Sie unterstützt einen maximalen Zählbereich von -2^{31} bis $+2^{31} - 1$ (entsprechend $-2\,147\,483\,648$ bis $+2\,147\,483\,647$) und erreicht je nach verwendeter Gebersignale eine maximale Eingangsfrequenz von bis zu 20 kHz pro Kanal [7].

Die Baugruppe ermöglicht verschiedene Betriebsarten: endloses, einmaliges sowie periodisches Zählen (jeweils vorwärts oder rückwärts), Frequenzmessung, Drehzahlmessung, Periodendauermessung sowie präzises Dosieren. Der Zählvorgang kann wahlweise über das Anwenderprogramm (Software-Tor) oder über externe Signale (Hardware-Tor) gestartet und gestoppt werden. Dabei können Zähl-, Tor- und Richtungssignale direkt an die Baugruppe angeschlossen werden.

Für jeden Zählkanal ist es möglich, einen Vergleichswert zu definieren. Im Dosiermodus lassen sich bis zu vier Vergleichswerte je Kanal verwenden. Wird der definierte Vergleichswert erreicht, kann automatisch ein zugeordneter Ausgang gesetzt oder zurückgesetzt werden, um direkt steuerungsrelevante Aktionen auszulösen oder Prozessalarme zu generieren.

Innerhalb der Betriebsarten „Einmalig zählen“, „Periodisch zählen“ und „Dosieren“ lassen sich zusätzlich Zählgrenzen innerhalb des maximalen Zählbereichs festlegen. Im Vorwärtsbetrieb beginnt die Zählung bei 0, der Endwert ist dabei frei wählbar. Im Rückwärtsbetrieb wird der Startwert definiert, während der Endwert bei 0 liegt.

Die FM 350-2 unterstützt pro Zählkanal bis zu vier Prozessalarme. Zwei davon können durch Flankenwechsel am Hardware-Tor ausgelöst werden, zwei weitere sind abhängig von der jeweils eingestellten Betriebsart. Im Dosierbetrieb können bis zu fünf spezifische Alarmerzeugungen erzeugt werden.

Für die Zählerfunktion können Signale unterschiedlicher Gebertypen verwendet werden. Zulässig sind ausschließlich prellfreie Geber, darunter 24-V-Inkrementalgeber (Gegentakt oder P-Schalter), 24-V-Impulsgeber mit Richtungspegel, 24-V-Initiatoren ohne Richtungspegel (z. B. Lichtschranken oder BERO Typ 2) sowie NAMUR-Geber nach DIN 19234. Die Signaltypen können an den Eingängen in Vierergruppen zwischen 24-V- und NAMUR-Signalen konfiguriert werden. Zu beachten ist, dass an für NAMUR konfigurierte Eingänge keine Signalspannungen über 8,2 V angelegt werden dürfen. An den Tor- und Richtungsseingängen sind ausschließlich 24-V-Signale erlaubt.

Zum Schutz vor Störungen sind alle Eingänge mit einem einheitlichen Eingangsfilter (RC-Glied) mit einer Filterzeit von 50 µs versehen. Schnelle Reaktionen auf Zählereignisse sind durch digitale Ausgänge realisierbar, wobei pro Kanal ein Ausgang und im Dosiermodus bis zu vier Ausgänge verfügbar sind. Diese können zählerstandsabhängig oder über Steuerbits angesteuert werden.

Das Verhalten der Baugruppe bei einem CPU-STOP ist parametrierbar. Es kann festgelegt werden, ob die Betriebsart fortgeführt oder abgebrochen wird und ob die digitalen Ausgänge ihren aktuellen Zustand beibehalten, auf Ersatzwerte gesetzt oder abgeschaltet werden. Dabei ist besondere Vorsicht geboten, da Ersatzwerte auch an nicht freigegebenen Ausgängen anliegen können, was unter Umständen zu gefährlichen Anlagenzuständen führt.

Auch bei Ausfall der Baugruppenversorgung zeigt die FM 350-2 ein differenziertes Verhalten. Wird sie über einen Standard-Rückwandbus betrieben, erkennt die CPU bei Spannungsverlust einen Peripherie-Zugriffsfehler, und die Baugruppe startet nach Spannungswiederkehr nicht automatisch neu. Beim Einsatz eines aktiven Rückwandbusses hingegen wird der CPU ein Ziehen-Alarm und bei Wiederkehr der Versorgungsspannung ein Stecken-Alarm gemeldet.

Zusätzlich verfügt die FM 350-2 über umfassende Diagnosefunktionen. Mögliche Fehlerzustände wie fehlerhafte Geberversorgung (NAMUR), fehlende oder fehlerhafte Parametrierung, Zeitüberwachungsfehler (Watchdog), Drahtbruch oder Kurzschluss sowie der Verlust von Prozessalarmen können erkannt und gemeldet werden [7].

3.2.1 Grundlegende Funktionen der FM 350-2

Die FM 350-2 bietet eine Vielzahl von Funktionen zur Steuerung und Überwachung von Zählprozessen. Hierzu gehören unter anderem das Initialisieren der Zähler, das Steuern von Digitalausgängen sowie das Laden und Auslesen von Zählwerten.

3.2.1.1 Steuerung der Baugruppe mit FC CNT2_CTR

(FC2)

Die Funktion `extttFC CNT2_CTR` ermöglicht die Steuerung der Digitalausgänge der FM 350-2 sowie die Verwaltung der Software-Tore. Zudem können Rückmeldesignale ausgelesen werden.

Hauptfunktionen:

- Initialisierung des Zähler-DBs
- Auslesen der Rückmeldesignale und Speicherung in der Struktur `CHECKBACK_SIGNALS`
- Übertragen der Steuersignale aus der Struktur `CONTROL_SIGNALS` zur FM 350-2

Die Funktion muss zyklisch (z. B. im OB1 oder im Weckalarm OB35 der S7-300) aufgerufen werden, um eine kontinuierliche Überwachung und Steuerung zu gewährleisten.

3.2.1.2 Laden von Zählerständen, Grenz- und Vergleichswerten (`extttFC3` / `extttFB3`)

Mit der Funktion `extttFC CNT2_WR` oder dem Funktionsbaustein `extttFB CNT2WRPN` können Zählerstände und Vergleichswerte neu geladen werden. Diese Funktion sollte nur verwendet werden, wenn im laufenden Betrieb neue Werte in das Modul eingespielt werden müssen.

3.2.1.3 Auslesen von Zähl- und Messwerten (extttFC4 / extttFB4)

Die extttFC CNT2_RD / extttFB CNT2RDPN Funktion dient zum zyklischen Auslesen der aktuellen Zählwerte. Falls keine Leseaufträge benötigt werden, kann auf diese Funktion verzichtet werden, um die Prozesslast zu minimieren.

3.2.1.4 Diagnosedaten lesen (FC5)

Im Falle eines Diagnosealarms ermöglicht die Funktion FC DIAG_RD das Laden der Diagnosealarmdaten in den Zähler-DB, um Fehlerursachen zu analysieren und entsprechende Maßnahmen einzuleiten.

3.2.2 Einschränkungen der FM 350-2

Trotz ihrer vielseitigen Einsatzmöglichkeiten weist die FM 350-2 einige Limitierungen auf:

- Begrenzte Flexibilität in der Anpassung an komplexe Steuerungsaufgaben
- Kein FPGA-basierter Aufbau, wodurch individuelle Anpassungen nicht möglich sind
- Begrenzte Echtzeitfähigkeit im Vergleich zu modernen Zählmodulen

3.3 Einsatzgebiete der FM 350-2

3.3.1 Haupteinsatzgebiete

Die FM 350-2 wird hauptsächlich dort eingesetzt, wo Signale gezählt und schnelle Reaktionen auf vorgegebene Zählerstände erforderlich sind. Besonders relevant ist sie für Anwendungen in der Fertigungs- und Automatisierungstechnik.

3.3.2 Typische Anwendungen

Typische Einsatzbereiche sind:

- Verpackungsanlagen
- Sortieranlagen
- Dosieranlagen
- Drehzahlregelungen und Überwachung von Gasturbinen

3.3.3 Anwendungsbeispiel: Kartonabfüllung

Ein typisches Beispiel für die Nutzung der FM 350-2 ist die Abfüllung von Teilen in Kartons:

- Kanal 0 zählt die Teile und steuert das Abfüllventil.
- Kanal 1 steuert den Transport der Kartons und erfasst die Anzahl der gefüllten Kartons.
- Das System stellt sicher, dass exakt die vorgegebene Anzahl an Teilen pro Karton abgefüllt wird.

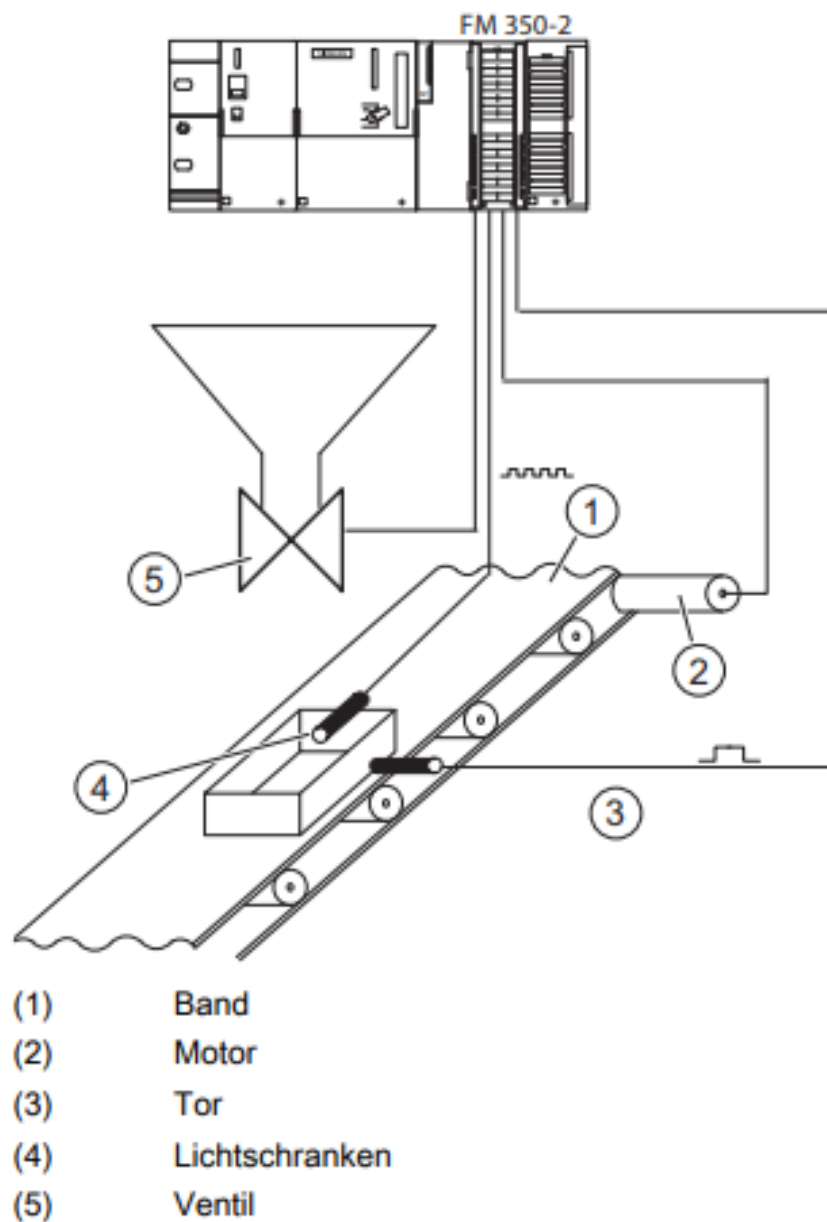


Abbildung 3.5: Beispiel für den Einsatz einer FM 350-2 in der S7-300 [7]

3.4 Vorteile und neue Möglichkeiten durch das TM FAST Modul

Das TM FAST Modul bietet im Vergleich zur FM 350-2 eine Reihe von Vorteilen und neuen Möglichkeiten, die es zu einer attraktiven Alternative machen. In die-

sem Abschnitt werden die wesentlichen Unterschiede und Vorteile des TM FAST Moduls beschrieben.

- **FPGA-basierte Architektur:** Die flexible und programmierbare Hardwareplattform ermöglicht eine Anpassung an spezifische Anforderungen und komplexe Steuerungsaufgaben.
- **Echtzeitfähigkeit:** Durch die moderne Architektur bietet das Modul eine verbesserte Echtzeitfähigkeit, die für anspruchsvolle industrielle Anwendungen entscheidend ist.
- **Erweiterte Funktionalität:** Neben Zähl- und Messfunktionen kann das TM FAST Modul Prozessalarme auslösen und direkt in den Produktionsprozess eingreifen.
- **Hohe Flexibilität:** Die programmierbare Architektur erlaubt eine einfache Erweiterung und Anpassung der Funktionalität, wodurch das Modul vielseitig einsetzbar ist.
- **Integration in moderne Steuerungssysteme:** Das Modul ist optimal auf die Anforderungen moderner Automatisierungssysteme abgestimmt und bietet eine nahtlose Integration.

RS485, 24V, userwriterec, FB-IF/CTRL und 5v verwendbar. Alle Möglichkeiten zur kanalvergabe und Datenübertragung (zyklisch und azyklisch erwähnen)

3.5 Anforderungen an die Neuentwicklung

Kriterium	TM FAST	FM 350-2
Unterschiede		
Architektur	FPGA-basiert, flexible und programmierbare Hardwareplattform	Reines Zählermodul ohne FPGA-Technologie
Funktionalität	Kann komplexe Prozesssteuerungs- und Überwachungsaufgaben übernehmen	Spezialisiert auf Zählen, Frequenz- und Drehzahlmessung
Prozessintegration	Kann direkt in den Produktionsprozess eingreifen und Prozessalarmlöser auslösen	Liefert Zähler- und Messwerte, die von der übergeordneten Steuerung weiterverarbeitet werden
Flexibilität	Hohe Flexibilität und Anpassungsfähigkeit durch FPGA-Architektur	Festgelegte Funktionalität, die nicht ohne Weiteres erweitert werden kann
Gemeinsamkeiten		
Zählfunktionen	Beide Module bieten Zählfunktionen mit hoher Auflösung (z.B. 32-Bit Zähltiefe)	
Frequenz- und Drehzahlmessung	Beide Module unterstützen die Messung von Frequenzen und Drehzahlen	
Einbindung in Steuerungsarchitektur	Beide Module werden in die übergeordnete Steuerungsarchitektur integriert	
Industrielle Anwendungen	Beide Module finden Einsatz in ähnlichen Industriebereichen wie Verpackung, Sortierung, Dosierung etc.	

Tabelle 3.4: Vergleich zwischen TM FAST und FM 350-2

4 Entwicklung des VHDL-basierten Applikationsbeispiels

Hier ist maybe wichtig Aspekte des Software Engineerings, Best Practices, Modularisierung und Wiederverwendbarkeit, Teststrategien und Debugging-Techniken

4.1 Konzept und Architektur des neuen Applikationsdesigns

Byte Zuweisung für FB_IF (32 Byte)

- 3 Byte für Kanalobergrenze = 4095 (Standartventil ist 1 Impuls/ 1 ml -> 4l abfüllung) => bis zu 10 Kanäle möglich
- 4 Byte für Kanalobergrenze = 65535 (65l abfüllung) => nur 8 Kanäle möglich

Andere Ideen: Vielleicht auf TMFASTUserWriteREC ausweichen? (Bis zu 128 Byte) asynchron (was ist wenn daten mitten im ablauf geändert werden?)

Algorithmus, der im ersten Zyklus die Kanalobergrenzen 1–4 von der CPU an das TMFast sendet, im zweiten Zyklus 5–8 usw. ist synchron und eignet sich besser die Steuerschnittstelle, da über die im Betrieb auch taktsynchron neue Werte geschrieben werden können. Der Nachteil des Write-Records ist die Asynchronität zum Zyklus.

Zähler -> Unter- und Obergrenze einstellbar (nur Obergrenze)

CPU Stop beachten:

Das umgesetzte Konzept besteht darus, die Obergrenze aller geladenden Kanäle (im bezug auf "dynamisches"laden der Kanalmänge -> wichtig, da sonst 1400 Logikelemente (16 DWord für 12 Kanäle) zur speicherung nutzlos verbraucht werden)

azyklisch über den TMFASTUserWriteREC zu verschicken. Bietet die Möglichkeit 1 bis 32 DWord (4 - 128 Byte) an Länge zu verschicken. Ein DWord (ca. 130 Logikelemente) reicht also für einen Kanal um eine 32 Bit Zahl zu speichern, die die Obergrenze des Kanals angibt. Eine 32 Bit Zahl ohne Vorzeichen (4294967295) ist für die meisten Anwendungen ausreichend, da sie 4.294.967.295 Impulse (z.B. 1 Impuls = 1 ml) darstellen kann, was 4294967295 ml oder 4294,967295 l entspricht. Die Steuerungswerte wie z.B. Reset, Load oder Hold werden über die zyklischen Schnittstellen CTRL_IF über 1 DWord große Bitmuster übertragen und die Statuswerte über die zyklischen Schnittstellen FB_IF zurückgegeben.

4.2 Überlaufkompensation (Dynamic Overflow Compensation)

Diese Methode wird für präzise Dosierungssysteme verwendet, bei denen der Nachlaufeffekt nach Ventilschließung berücksichtigt werden muss.

4.2.1 Vorteile

- Berücksichtigung der Systemträgheit (mechanische Nachlaufzeit des Ventils)
- Kompensation des systematischen Fehlers durch den Nachlauf
- Bei konstanten Bedingungen (Druck, Temperatur, Viskosität) sehr genau
- Keine zusätzliche Hardware erforderlich
- Selbstlernend/adaptiv, wenn der letzte Überlauf für die nächste Dosierung verwendet wird
- Gut für kontinuierliche Prozesse geeignet

4.2.2 Nachteile

- Bei schwankenden Bedingungen (Druck, Temperatur, Viskosität) kann der Überlauf variieren
- Die erste Dosierung nach einem Neustart hat keine Überlaufkompensation

- Bei Störungen (z.B. Luftblasen) wird ein falscher Überlaufwert für die nächste Dosierung verwendet
- Bei Produktwechsel muss das System neu einlernen
- Bei sehr unterschiedlichen Füllmengen muss eventuell mit verschiedenen Überlaufwerten gearbeitet werden

4.2.3 Alternative Methoden

1. Prädiktion des Abschaltzeitpunkts

- Berechnung basierend auf aktueller Durchflussrate
- Berücksichtigung der Ventil-Schließzeit

2. Zweistufige Dosierung

- Schnelle Befüllung (ca. 90%)
- Langsame Präzisionsbefüllung

3. PID-geregelte Dosierung

- Kontinuierliche Regelung
- Hohe Präzision

4. Massendurchflussmesser

- Direkte Massenmessung
- Höchste Genauigkeit

4.2.4 Mathematische Beschreibung

Für die Überlaufkompensation gilt:

$$V_{target} = V_{desired} - V_{overflow_previous} \quad (4.1)$$

Wobei:

$$V_{target} = \text{Ziel-Abschaltvolumen}$$

$$V_{desired} = \text{Gewünschtes Endvolumen}$$

$$V_{overflow_previous} = \text{Gemessener Überlauf der vorherigen Dosierung}$$

4.3 Analyse der Überlaufkompensation

Eine zusätzliche Berechnung der Überlaufkompensation ist nicht vorteilhaft, da das System bereits eine optimale Implementierung aufweist.

4.3.1 Aktuelle Implementierung (automatisch)

- **Kontinuierlicher Zählvorgang:**

$$Count_{total} = Count_{target} + Count_{overflow} \quad (4.2)$$

- **Ventilsteuerung:**

$$Valve_{state} = \begin{cases} 0 & \text{wenn } Count_{current} \geq Count_{target} \\ 1 & \text{wenn } Count_{current} < Count_{target} \end{cases} \quad (4.3)$$

- **Automatische Nachlaufkompensation:**

- Nachlaufmenge wird direkt zum nächsten Zyklus addiert
- Keine Zwischenspeicherung notwendig
- Keine Berechnungszeit erforderlich

4.3.2 Vorteile der automatischen Implementierung

1. Effizienz:

- Keine zusätzlichen Berechnungsschritte
- Keine Verzögerungen im Prozessablauf
- Optimale Ressourcennutzung

2. Genauigkeit:

- Keine Rundungsfehler durch Berechnungen
- Direkte Messung statt berechneter Werte
- Kontinuierliche Erfassung ohne Unterbrechung

3. Zuverlässigkeit:

- Reduzierte Komplexität
- Weniger potenzielle Fehlerquellen
- Robustes Systemverhalten

4.3.3 Nachteile einer zusätzlichen Berechnung

- **Erhöhte Komplexität:**

- Zusätzliche Prozesse und Variablen
- Komplexere Wartung und Debugging
- Schwierigere Systemanalyse

- **Potenzielle Fehlerquellen:**

- Überläufe bei arithmetischen Operationen
- Timing-Probleme bei Berechnungen
- Synchronisationsfehler zwischen Prozessen

- **Ressourcenverbrauch:**

- Zusätzlicher Speicherbedarf
- Erhöhte CPU-Last
- Mehr FPGA-Ressourcen benötigt
- **Keine Genauigkeitsverbesserung:**
 - Keine Verbesserung der Messgenauigkeit
 - Mögliche Verschlechterung durch Rundungsfehler
 - Unnötige Komplexität ohne Mehrwert

4.3.4 Mathematische Betrachtung

Die Gesamtmenge einer Dosierung lässt sich beschreiben als:

$$V_{total} = V_{target} + V_{overflow} \quad (4.4)$$

Wobei im automatischen System gilt:

$$V_{next_target} = V_{desired} - V_{overflow_previous} \quad (4.5)$$

Dies wird implizit durch den kontinuierlich laufenden Zähler realisiert, ohne explizite Berechnung.

Lassen Sie uns das klären:

RS485 als physikalische Schnittstelle vs. Protokoll: RS485 ist ein Standard für die elektrische Übertragung von Signalen über eine differentielle Leitung. Das bedeutet, dass das Signal nicht als Spannung gegen Masse, sondern als Spannungsdifferenz zwischen zwei Leitungen (z.g. A und /A) übertragen wird. Dies macht die Übertragung sehr robust gegenüber elektromagnetischen Störungen, besonders über längere Distanzen. Protokolle (wie Modbus RTU, USS, Profibus DP, SSI, EnDat etc.) sind die Regeln, wie Daten über diese physikalische Schnittstelle strukturiert und interpretiert werden. Inkrementelle Geber und RS485-ähnliche Signale: Viele inkrementelle Geber, insbesondere solche mit HTL-Differentiell (High-Transistor Logic, 24V) oder TTL-Differentiell (Transistor-Transistor Logic, 5V)

Ausgängen, nutzen die elektrischen Eigenschaften von RS485 für ihre A/B/N-Spuren. Das bedeutet, sie geben für jede Spur ein differentiellles Signalpaar aus (z.B. A und /A, B und /B, N und /N). Diese Geber senden jedoch KEIN seriellles Kommunikationsprotokoll. Sie senden einfach hochfrequente Rechteckimpulse auf diesen differentiellen Leitungen. Wie die TM-FAST (oder EncEmu) das verarbeitet: Module wie die TM-FAST oder die EncEmu TM155E sind Hochgeschwindigkeitszähler. Sie sind genau dafür ausgelegt, diese differentiellen Impulse zu empfangen. Sie erkennen die Flankenwechsel (z.B. von $A > /A$ auf $A < /A$ und umgekehrt) auf den A- und B-Spuren. Durch die Phasenverschiebung zwischen den A- und B-Spuren (typischerweise 90 Grad) können sie nicht nur die Anzahl der Impulse zählen, sondern auch die Drehrichtung des Gebers erkennen. Die N-Spur (oder Z-Spur) liefert einen Referenzimpuls pro Umdrehung zum Nullpunkt. Es wird kein Software-Protokoll auf der SPS benötigt, um diese Impulse zu interpretieren. Das Modul selbst (TM-FAST, EncEmu) übernimmt das Zählen und die Richtungserkennung auf Hardware-Ebene und stellt der SPS dann den aktuellen Zählerstand zur Verfügung. Zusammenfassend:

Ihre Beschreibung "wenn eine 1 anliegt (also 5,9v oder so) wird wie bei den di ein impuls wargenommen und um 1 inkrementiert"passt perfekt zur Funktionsweise eines inkrementellen Gebers, der differentielle Signale liefert. Die 5,9V würden hier die Spannungsdifferenz zwischen den beiden Leitungen des Paares darstellen.

Fazit:

Ja, es ist absolut möglich und der vorgesehene Weg, inkrementelle Geber mit differentiellen Ausgängen (die RS485-ähnliche elektrische Eigenschaften nutzen) an einer SPS mit entsprechenden Hochgeschwindigkeits-Eingangsmodulen wie der TM-FAST oder der EncEmu TM155E zu betreiben. Diese Geber funktionieren über reine Impulsausgabe und nicht über serielle Kommunikationsprotokolle. Protokolle kommen typischerweise bei Absolutwertgebern (z.B. SSI, EnDat, Profinet-Geber) zum Einsatz, die eine absolute Position über eine serielle Datenleitung übertragen.

/textbf(Warum die Kanalbelegung? RS485 Eingänge sind Strong deshalb mehr)

4.4 Umsetzung der Funktionalität in VHDL

-Diagramme einfügen, beschreiben und analysieren

4.5 Test und Validierung der VHDL-Implementierung

-Questa simuliert Eingänge und prüft die Ausgänge

4.6 Schnittstelle mit dem TIA Portal

-Diagramm für Schnittstelle und Funktion der TIA Anwendung

4.7 Vergleich mit der bisherigen Lösung

-Auf abschnitte in der Analyse eingehen

5 Fazit und Ausblick

5.1 Zusammenfassung der Ergebnisse

5.2 Kritische Reflexion und Herausforderungen

5.3 Potenzielle Weiterentwicklungen und zukünftige Anwendungen

Literaturverzeichnis

- [1] Ernst Klett Verlag (2004): *Infoblatt Siemens AG*. Verfügbar unter: <https://www.klett.de/alias/1036900>. Zugriff: 5. Januar 2025.
- [2] Merkur (2022): *Siemens – Geschichte, Aktie und Tätigkeitsfelder*. Verfügbar unter: <https://www.merkur.de/wirtschaft/siemens-geschichte-aktie-und-taetigkeitsfelder-91268844.html>. Zugriff: 19. Januar 2024.
- [3] Siemens, 2023, zitiert nach de.statista.com: *Umsatz von Siemens SE seit 2005*. Verfügbar unter: <https://de-statista-com.thn.idm.oclc.org/statistik/daten/studie/73827/umfrage/umsatz-von-siemens-seit-2005/>. Zugriff: 5. Januar 2025.
- [4] ResearchGate: Verfügbar unter: https://www.researchgate.net/figure/SA-88s-physical-model-and-an-example-production-allocation_fig2_308384237/. Zugriff: 11. Januar 2025.
- [5] Siemens (2022): *Pflichtenheft-SmartFactory*. Internes Dokument.
- [6] Liam Bee (2022): *PLC and HMI development with Siemens TIA Portal: develop PLC and HMI programs using standard methods and structured approaches with TIA Portal V17*. Packt Publishing, Limited.
- [7] Siemens AG (2011): *Zählerbaugruppe FM 350-2 Gerätehandbuch*. Verfügbar unter: https://cache.industry.siemens.com/dl/files/178/1105178/att_20226/v1/s7300_fm350_2_operating_instructions_de_de-DE.pdf. Zugriff: 11. März 2025.
- [8] Siemens AG (2023): *TMFast Programmeier Handbuch*. Verfügbar unter: https://cache.industry.siemens.com/dl/files/088/109816088/att_1144842/v2/s71500_tm_fast_programming_manual_de-DE_de-DE.pdf. Zugriff: 12. März 2025.

- [9] IPH - Institut für Integrierte Produktion Hannover gGmbH: *Automatisierungspyramide: Definition, Bedeutung, Ebenen*. Verfügbar unter: <https://www.iph-hannover.de/de/dienstleistungen/automatisierungstechnik/automatisierungspyramide/>. Zugriff: 26. Juli 2025.
- [10] H. Berger: *Automatisieren mit SIMATIC S7-1500: Projektieren, Programmieren und Testen mit STEP 7 Professional*. 2. Aufl. Erlangen: Publics Publishing, 2017.
- [11] W. Babel: *Systemintegration in Industrie 4.0 und IoT: Vom Ethernet bis hin zum Internet und OPC UA*. Wiesbaden: Springer Fachmedien, 2024. DOI: <https://doi.org/10.1007/978-3-658-42987-4>.
- [12] Siemens AG: *Totally Integrated Automation Portal*. siemens.com. Verfügbar unter: <https://new.siemens.com/de/de/produkte/automatisierung/industrie-software/automatisierungs-software/tia-portal.html> (abgerufen am 26.07.2025).
- [13] Siemens AG: *SIMATIC Energy Suite*. siemens.com. Verfügbar unter: <https://www.siemens.com/de/de/produkte/automatisierung/industrie-software/automatisierungs-software/energiemanagement/simatic-energy-suite.html> (abgerufen am 26.07.2025).
- [14] Siemens AG: *SIMATIC ET 200MP*. siemens.com. Verfügbar unter: <https://new.siemens.com/de/de/produkte/automatisierung/systeme/industrie/io-systeme/simatic-et-200mp.html> (abgerufen am 26.07.2025).
- [15] Intel® Corp.: *Intel Cyclone 10 LP Device Overview*, 2022. [Online] Verfügbar unter: <https://www.intel.com/content/www/us/en/docs/programmable/683879/current/device-overview.html> (abgerufen am 27.07.2025).
- [16]

6 Plan

Hier sind einige Schritte, die Ihnen dabei helfen können, eine VHDL-Applikation für das TM FAST-Modul zu entwickeln, um das alte FM350-2-Modul zu ersetzen:

Anforderungsanalyse

- Analysieren Sie sorgfältig die Funktionalität und Schnittstellen des FM350-2-Moduls.
- Identifizieren Sie die Kernfunktionen, die das neue TM FAST-Modul erfüllen muss.
- Definieren Sie klar die Eingangs- und Ausgangssignale, die Kommunikationsprotokolle und andere relevante Spezifikationen.

Architekturentwurf

- Entwerfen Sie eine modulare Architektur für Ihre VHDL-Applikation.
- Unterteilen Sie die Funktionalität in logische Blöcke wie Steuerungslogik, Kommunikationsschnittstellen, Datenpfade usw.
- Legen Sie die Schnittstellen zwischen den einzelnen Modulen fest.

VHDL-Entwicklung

- Beginnen Sie mit der Implementierung der einzelnen VHDL-Module basierend auf Ihrem Architekturentwurf.
- Achten Sie auf eine strukturierte und modulare Codeerstellung, um die Wartbarkeit und Erweiterbarkeit zu erleichtern.
- Verwenden Sie VHDL-Sprachkonstrukte wie Prozesse, Signale, Typen und Pakete, um Ihre Applikation übersichtlich und effizient zu gestalten.

Simulation und Test

- Erstellen Sie umfangreiche Testbänke, um Ihre VHDL-Applikation gründlich zu überprüfen.
- Führen Sie Simulationen durch, um das korrekte Verhalten der einzelnen Module und der Gesamtapplikation zu verifizieren.
- Testen Sie die Applikation schrittweise, um mögliche Fehler frühzeitig zu erkennen und zu beheben.

Integration und Validierung

- Integrieren Sie Ihre VHDL-Applikation in die Gesamtarchitektur des TM FAST-Moduls.
- Testen Sie die Applikation in der realen Umgebung, um ihre Kompatibilität und Funktionalität zu validieren.
- Passen Sie die Applikation gegebenenfalls an, um Anforderungen und Spezifikationen des TM FAST-Moduls zu erfüllen.

Dokumentation und Versionskontrolle

- Erstellen Sie eine ausführliche Dokumentation, die die Architektur, Schnittstellen, Konfiguration und Verwendung Ihrer VHDL-Applikation beschreibt.
- Nutzen Sie ein geeignetes Versionskontrollsystem, um Änderungen an Ihrer Applikation nachzuverfolgen und zu verwalten.

Durch diese strukturierte Vorgehensweise können Sie sicherstellen, dass Ihre VHDL-Applikation für das TM FAST-Modul die Anforderungen des FM350-2-Moduls erfüllt und in die Gesamtarchitektur des TM FAST-Systems nahtlos integriert wird.

7 Fragen

Müssen Alarme umgesetzt werden? -> Müssen nicht berücksichtigt werden
Müssen Hardwareconfigs von der CPU oder von TMFast gehandhabt werden? -> hardwareconfig
der (z.B. Ventile) ist in VHDL (andere Kommunikationsprotokolle und co pro z.B.
Ventile) Wie verhalten sich die In- und Outputs?

- Sollen alle Eingänge Zähl- und Richtungseingang sein oder Aufteilung zwischen den beiden? Nur zähler
-

wie funktionieren ventile überhaupt?

bit an die cpu senden

nachlauf bei inkrement zum abfüllen

cpu macht - tm fast macht ventil zu

auf ist 1 zu ist low für ventil

abfüllende menge muss übergeben werden

vhdl auf hoher ebene darstellen (als baustein)

synchron oder asynchron (für zähler oder generell) -> Alles Synchron (TM Fast arbeitet (einstellbar mit 50mHz) schneller als ein ventil) -> dadurch zählt es schneller als inkremente des ventils

clk wird bei änderung aktiviert (High->Low / Low->High)

steuerung des ventils über PLC oder TMFAST? Erstmal Plc mit bool senden (überlauf für counter (justierung)) / TMFast erfordert Kommunikations-Protokolle
Filter?

Anwendung in mehreren oder einem Process in vhdl?

soll der timer nur zählen, wenn die flanke positiv wird? oder auch wenn sie negativ wird? -> nur bei positiv

Allgemein: Schriftgröße und Seitenränder? Zitieren von Handbüchern (Graue Literatur) und ChatGPT? Mit Disclaimer

ES IST KEIN PRAXISBERICHT (KEIN PROZESS BESCHREIBEN) / EVTL: VORNACHTEILANALYSE

Vielleicht auf WR_REC ausweichen, wenn FB_IF nicht groß genug?

Muss der Siemens Betreuer einen Bachelor haben? Was kommt auf den Betreuer zu (Organisatorisch)

Steuerung eines Ventils von TMFAST? Über DQ -> Wie und was wird dafür benötigt, reicht es ein signal zu senden?

Ventile 1. Inkrementalgeber mit zwei Spuren (A/B-Signal): Viele Durchflussmesser oder Antriebssysteme (z.B. Drehgeber) geben zwei phasenversetzte Impulssignale aus: Spur A und Spur B.

Anhand der Phasenlage erkennt die CPU die Richtung:

Wenn A vor B kommt → Vorwärts

Wenn B vor A kommt → Rückwärts

Diese Technik nennt sich Quadraturauswertung. 24 V (HTL) sind Standardventil muss aber auch andere können:

nur die Anzahl der Impulse wichtig? 12 Kanäle sind nur 6 Alternativen? RS485 für Impulse -> JA? TTL-Signals Transistor-Transistor-Logik?

Willi: A/B oder A/B/N Geber? RS422? 24V für DI und 5V für CH. Anforderungen: Gleiche Technologie? flexibel (wie umsetzen)? Nur einen Eingang (A) war bei 350-2 mit (A/B) und 24V (DI) als Eingang

Maybe:

- A -> 5V (RS-485) * 8 / (DI) 24V * 8 / (DIQ) 24V * 4 / (DQ) 24V * 8
- A/B -> 5V * 4 / (DI) 24V * 4 / (DIQ)? 24V * 2
- A/B/N -> 5V * 2 / (DI+DIQ?) 24V * 3

+CH0 kann nicht beides verwendet werden -> EIA-485, auch als RS-485, benutzt ein Leitungspaar, um den invertierten und einen nichtinvertierten Pegel eines 1-Bit-Datensignals zu übertragen. Am Empfänger wird aus der Differenz der beiden Spannungspegel das ursprüngliche Datensignal rekonstruiert

wo finde ich noch informationen zum software reset (ResetSW)?

Wie funktionieren RS422/RS485 im vergelich zu DI in VHDL? Erweiterung auf mehrere Kanäle? In die vorhandenen oder als neue Prozesse? Wie Initialisiert man andere Kanäle dynamisch? Um Speicherplatz zu sparen. link in logikgruppe Gibt es einen Counter/Timer den man auf 1 Sekunde stellen kann? frequenzmesser/perioden messung

8 Wichtig

Effectuated Interfaces

Control interface: control interface, direction: from CPU to TM FAST

Feedback interface: feedback interface, direction: from TM FAST to CPU

Read record: asynchronous read record data from TM FAST to CPU

Write record: asynchronous write record data from CPU to TM FAST

Anforderungen: Vergleichswertübergabe azyklische über UserWriteREC 8x 24V Geber (evtl. auf 12 Upgraden mit DIQ) CounterReset -> Setze Counter=0 CounterHold -> Ignoriere Geber-Impulse ResetSW -> Setzt MAX_VALUE=0 und Counter=0 Load -> Setzt MAX_VALUE und Counter=0 (Muss, weil wenn MAX_VALUE > Counter kein Fehler (Der nutzer darf dafür nicht einbüßen))

- Es gibt 12 Kanäle. Ich würde jeweils die Eingänge und Ausgänge aus der gleichen Leistungsklasse verknüpfen (DI-DQ; Umschaltbare DIQ; RS485)
- Der Sollwert, an dem abgeschaltet werden soll, wird über den Anwenderdatensatz übermittelt, Datensatz mit 32 Byte, 2 Byte Kennung und dann 14 mal der Sollwert als Integer (2 byte). Oder direkt in der Steuerschnittstelle. Der Datensatz kann im Betrieb nachgeladen werden mit allen Werten.
- Die Eingänge zählen jeweils von 0 bis zum eingestellten Sollwert und dann nahtlos wieder bei 0.
- Die Ausgänge werden mit einer Flanke in einem Bit pro Kanal in der Steuerschnittstelle gestartet und durch das Erreichen des Sollwerts wieder zurückgenommen. Es ist mit einem gemeinsamen Parameter für alle Kanäle einstellbar, ob ein Ausgang mit high aktiv ist oder mit low. Wird vor Erreichen des Sollwerts das Bit in der Steuerschnittstelle (SS) zurückgenommen wird auch der Ausgang wieder deaktiviert

- In der Rückmeldeschnittstelle wird der Zustand jedes Ausgangs angezeigt. Zusätzlich für jeden Kanal der aktuelle Zählwert oder die aktuelle Durchflussgeschwindigkeit angezeigt. Der Wert wird über ein Steuerbit in der Steuerschnittstelle ausgewählt. Der Wert für die Durchflussgeschwindigkeit könnte auf Ink/10s hochgerechnet werden, um eine entsprechende Auflösung zu erreichen. Ein bit in der SS zeigt an, welcher Wert gemeldet wird.

9 TODO

->A/B einbauen + testfall + andere testfälle (Nur A) ->Kanalbelegungen nachfragen (erstmal nur DI/DIQ) ->Grenzwertüberschreitung von enc_counter, wenn upperlimit < enc_counter mit in betracht ziehen. Reset wenn wert änderung bei upper_limit? Muss nicht, ist gewolltes Verhalten, auch nicht prüfen

->Reset über CTRL_IF (Sollte es Upper_Limit zurücksetzen?) ->UserWriteREC für die Obergrenzen einbauen (davor über CTRL_IF0(31 downto 16)/ Neue Anforderungen: Obergrenze muss nicht im Betrieb geändert werden) -> von 2 Byte auf 4 Byte durch mehr Speicher möglich ->Counterreset und Enable auf bits zuweisen (ähnlich wie reset) ->Test für overflowdetection ist useless

->UserWriteREC simulieren und Testen und mit fsm stabiler machen ->load,reset,hold logik überarbeiten(wann wird was ausgeführt und wodurch -> Muss nach einem Reset ein Load kommen um upperlimit neu zu laden (schaue beim obrgiegen simulieren)?)

Feedback gespräch -Diagramm für Ablauf -was hab ich mir gedacht -rst einbauen -Anforderungen mit aufschreiben

-Fragen dos donts ->Fault and diagnostic evaluation? -> DQ Force = 0 oder auch noch FB_IF? ->asynchronous signal WR_REC_NEW to the CLK domain ->CTRL_IF Signale auf variable zuweisen oder direkt verwenden? alias verwenden

Antwort: ->CTRL_IF für fehler quittieren, wenn overflow zu schnell toggelt ->ganzes paket ins netzwerklaufwerk des teams und diagramm und link des Product backlog ->generate für weniger logik (weniger kanäle/flexibel)

tobi berichten, dass durch test es sich etwas nach hinten verschiebt

->Review des Codes ->Quartus Compilieren und zum test TGEN (Am Ende)-> Test und angeschlossenden Enc_Enum (Gerät Geber-Simulation) (Davor evtl mit sich selbst DQ auf DI) ->Nightly Test einbauen

->wird wr_rec00 richtig gesetzt? testen

-FB_IF Finalform + Steuerbit -Load new and better Version

->RS422 für inc_enc einbauen

10 Tests

rückgabe über das (fb_if) feedback_interface

EncEmu encoder für impulse

schwierigkeit: di1 oder di0 zählen

RD_Rec sind azyklisch und nur selten benutzt

library test 8 counter -> name mit test cases Jürgen, ..., Kollege aus Prag

Muss ich nicht allein schreiben

Mathias fragen wenn funktionierendes Programm -> in den Nightly test muss in den nightly build

-systemlogic updaten auf v2 / DONE -zähler des sea teams verwenden / -Questa Simulation / (Questa DUT=Design under Test)

- 0000_0000_0000_0000_0000_0000_0000_0001 -> FB_IF(0)(0)<='1'; - /-> Q0.0 (FB_IF(0))

- FB_IF(1)(0) == QD4

-mathias jupiter.tfs.siemens.net für nightly build (muss ins git)

-adress register verbessern

-quellcode verwaltung?

Reaktion auf falsche Parameter Verifikation der korrekten Initialisierung des Encoders nach LOAD Grundkonfiguration und Initialisierung -> System-Reset wird ausgelöst

Verschiedene Zählmodi oder Edge-Typen?

- Zählen mit A oder A&B oder A&B&N -> nicht nötig

- Betriebsart: periodisches Zählen, einmaliges Zählen, usw. -> nicht nötig
- hoch/runterzählen, Invertieren der Hauptzählrichtung -> nicht nötig
- Test des Verhaltens beim Überlauf an Zählgrenzen (oben/unten) -> abhängig von Betriebsart -> Überlauf ist getestet/ Betriebsart ist unwichtig, da nur Hochgezählt wird (B=Low)
- Start nicht mit Wert 0 sondern Startwert -> Machbar aber muss aber nicht
- invertieren der A/B/N Eingangssignale -> ?
- Test der verschiedenen Resetquellen -> Muss

undefinds werden kopiert -> am start muss next_upper_limit initialisiert werden mit einer 0, sonst wird bei jedem load bevor WR_REC_NEW gemacht wird, next_upper_limi auf undefiend gesetzt.

11 Passwörter

PLC: Siemens123

- [Masterarbeit von Andreas Kick](#)
- [Bachelorarbeit von Thomas Weisel](#)
- [Prof. Meitner Zoom Link](#)
- [Enc_Emu](#)